

Bloom Filter

COMPCSI 752: Big Data Management

University of Auckland

(Credits to Ninh Pham)

Parts of this material are modifications of the lecture slides from

<http://mmds.org>

Designed for the textbook **Mining of Massive Datasets**

by Jure Leskovec, Anand Rajaraman, and Jeff Ullman

Auckland, May 2025

Basic definitions

- Let $\mathbf{U}$ be a universe of size n , i.e. $\mathbf{U} = \{1, 2, 3, \dots, n\}$ 全集
- Cash register model stream:
 - Sequence of m elements $a_1, \dots, a_m$ where $a_i \in \mathbf{U}$
 - Elements of $\mathbf{U}$ may or may not occur once or several times in the stream
 - Example: A sequence of email addresses, a sequence of IP addresses, ...
- Filtering a data stream (today's lecture):
 - Select elements with property x from the stream
满足某些性质

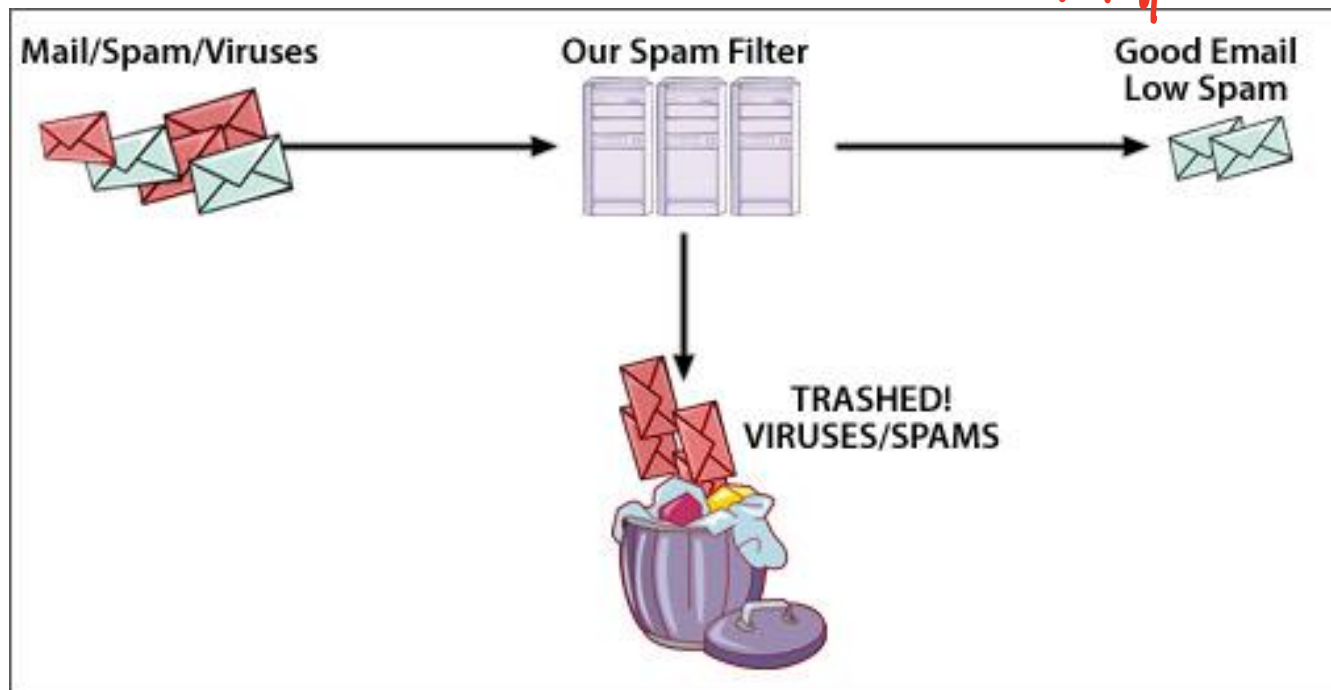
Filtering data stream

- Given a list of elements S , determine which stream elements are in S
- Obvious solution: Hash table
 - We do not have enough memory to store all S in a hash table
 - We might process **millions of filters** on the same stream, i.e. we have millions of different sets S

Applications

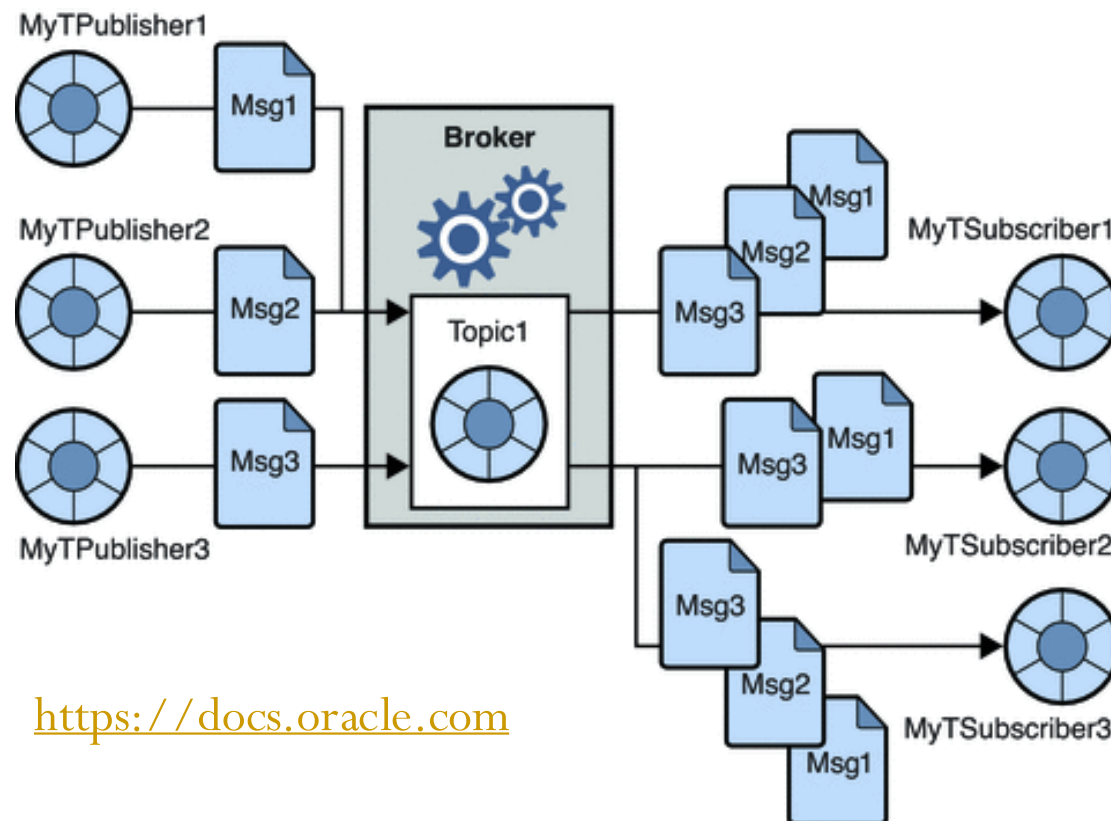
- Email spam filtering:
 - We know approximately **1 billion** “good” email addresses
 - If an email comes from one of these, it is NOT spam

刷屏



Applications

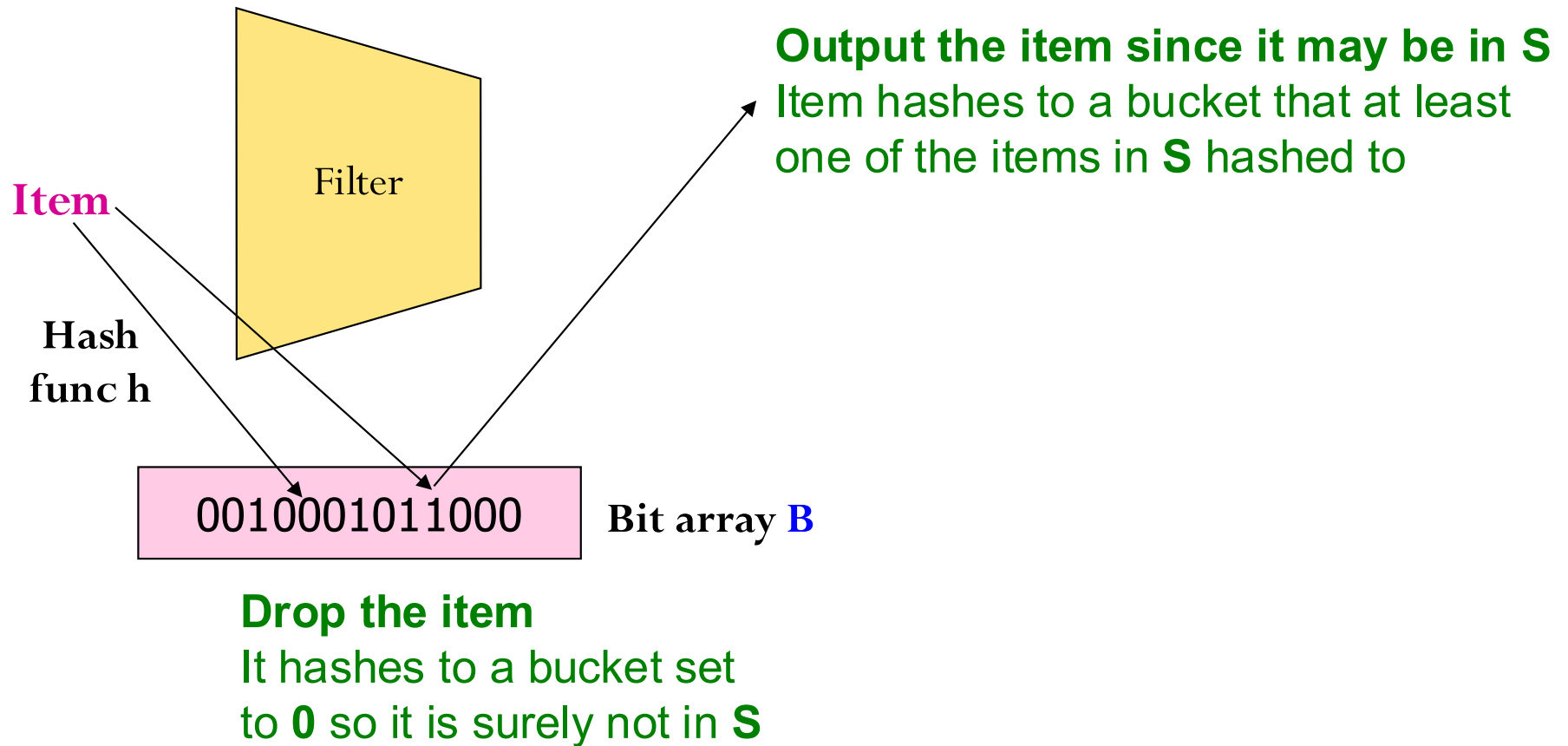
- Publish-subscribe systems:
 - You are collecting a lots of message (new articles)
 - People express interest in certain sets of keywords
 - Determine whether each message matches user's interest



<https://docs.oracle.com>

First cut solution

- Given a set of keys S
- Create a bit array $\mathbf{B}$ of n bits, initially at 0s
- Choose a hash function h with range $[1, n]$
- Hash each member $s \in S$ to one of n buckets, and set that bit to 1, i.e. $\mathbf{B}[h(s)] = 1$
- Hash each element a of the stream and output only those that hash to bit that was set to 1
 - Output a if $\mathbf{B}[h(a)] = 1$



- Create false positives but no false negatives:
 - If the item is in **S**, we surely output it (no false negatives)
 - If not, we may still output it (false positives)

Parameter settings

- $|S| = 1$ billion email addresses
- $|B| = 1 \text{ GB} = 8$ billion bits

Broom filter

- If the email is in S , then it hash to bucket that set to 1. It always gets through (no false negatives)
- Approximately $1/8$ of the bit are set to 1. So about $1/8$ of the email addresses not in S get through to the output (false positives)
 - Often, less than $1/8$, since we might have collisions, i.e. more than one email address hash to the same position

Analysis: Balls and bins model

- We want to analyze the number of false positives
- **General problem:** If we randomly throw m balls into n bins, what is the probability that a bin get at least one ball?
- **Our case:**
 - Balls = elements of S
 - Bins = buckets
 - Randomly throwing = hashing

Analysis: Balls and bins model

- We have m balls, n bins
- Prob. that a bin does not have any ball: $(1-1/n)^m$
- Prob. that a bin has at least one ball: $1-(1-1/n)^m$
- Since $(1-1/n)^m \sim e^{-m/n}$, the prob. that a bin gets at least one ball is: $1 - e^{-m/n}$
- Fraction of 1s in the array $\mathbf{B}$ = prob. of false positives
 $= 1 - e^{-m/n}$
- **Example:** $m = 1$ billions, $n = 8$ billion bits = 1GB, the fraction of 1s in $\mathbf{B}$ is 0.1175 ($\sim 1/8 = 0.125$)

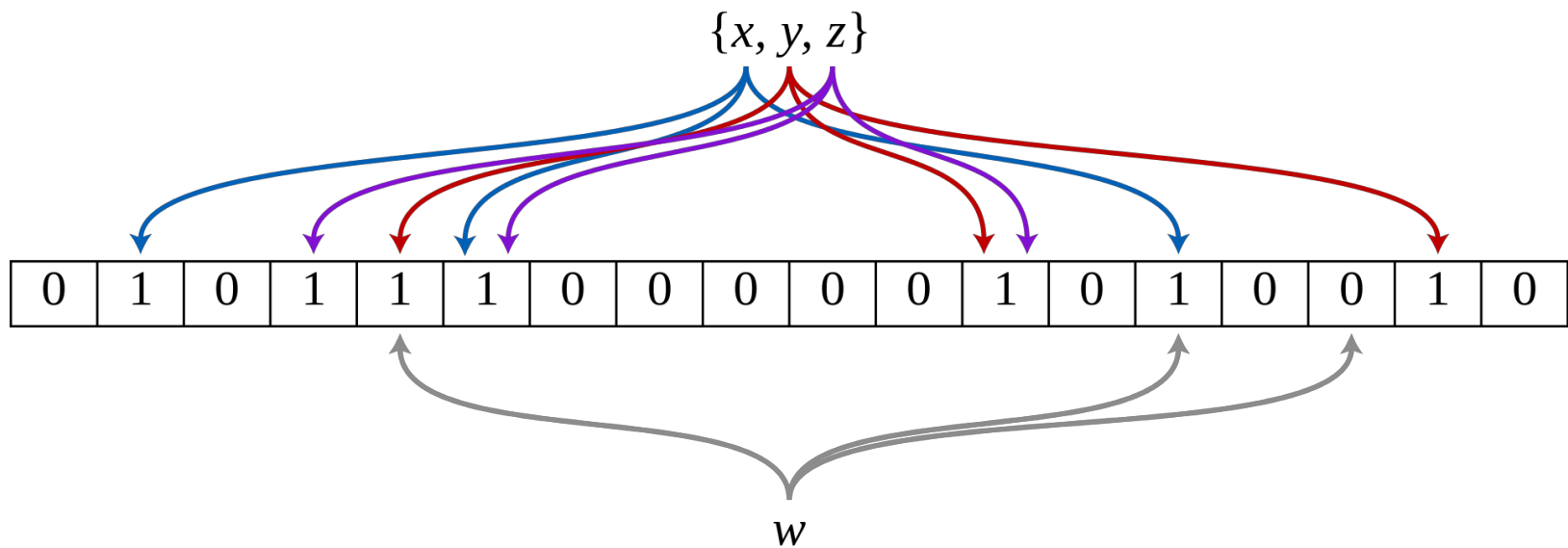
Bloom filter

- $|S| = m, |B| = n$
- Use k independent hash function $h_1, \dots, h_k$ with range $[1, n]$ k个 hash 一起用
- Construct Bloom filter:
 - Set B to all 0s
 - Hash each member $s \in S$ to one of n buckets using k hash functions, and set $B[h_i(s)] = 1$ for $i = 1, \dots, k$
- Filtering:
 - When a stream element a arrives,
 - If $B[h_i(a)] = 1$ for all $i = 1, \dots, k$, then declare $a \in S$
 - Otherwise, discard the element a

Bloom filter

- Example:

- $|S| = 3, |B| = 18$
- $S = \{x, y, z\}$
- $k = 3$
- Discard w since all $B[h(w)]$ are not **all** 1s

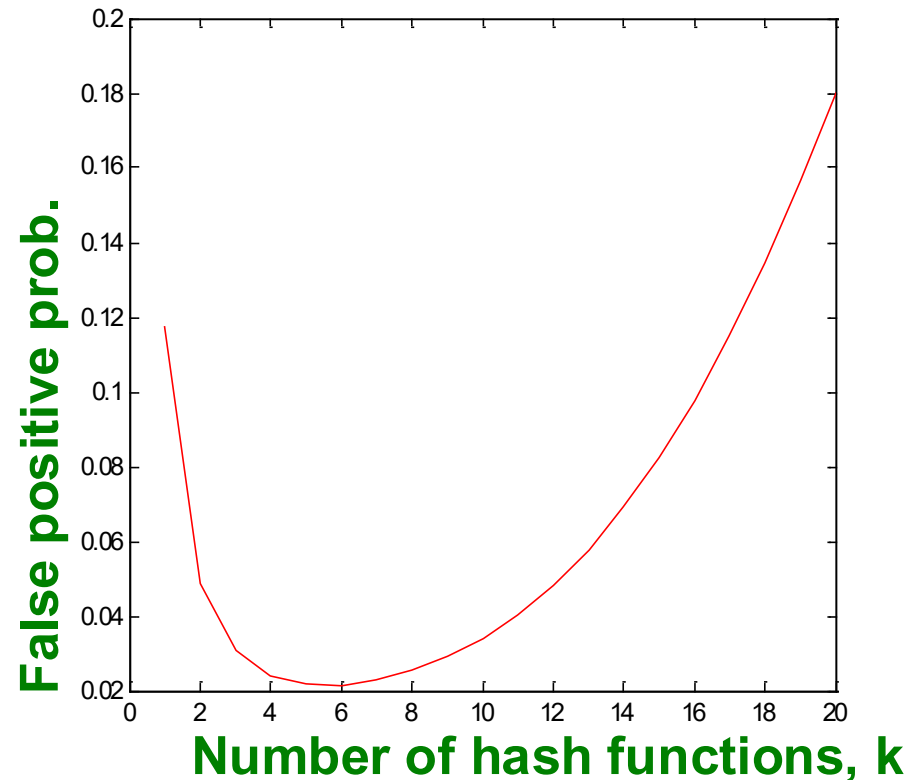


Bloom filter analysis

- What fraction of the bit vector **B** are 1s?
 - Randomly throw **km** balls into **n** bins
 - So fraction of 1s is $1 - e^{-km/n}$
- Prob. that 1 position of **a** is 1s: $1 - e^{-km/n}$
- Prob. that all **k** positions of **a** is 1s: $(1 - e^{-km/n})^k$
- So, false positive probability is: $(1 - e^{-km/n})^k$

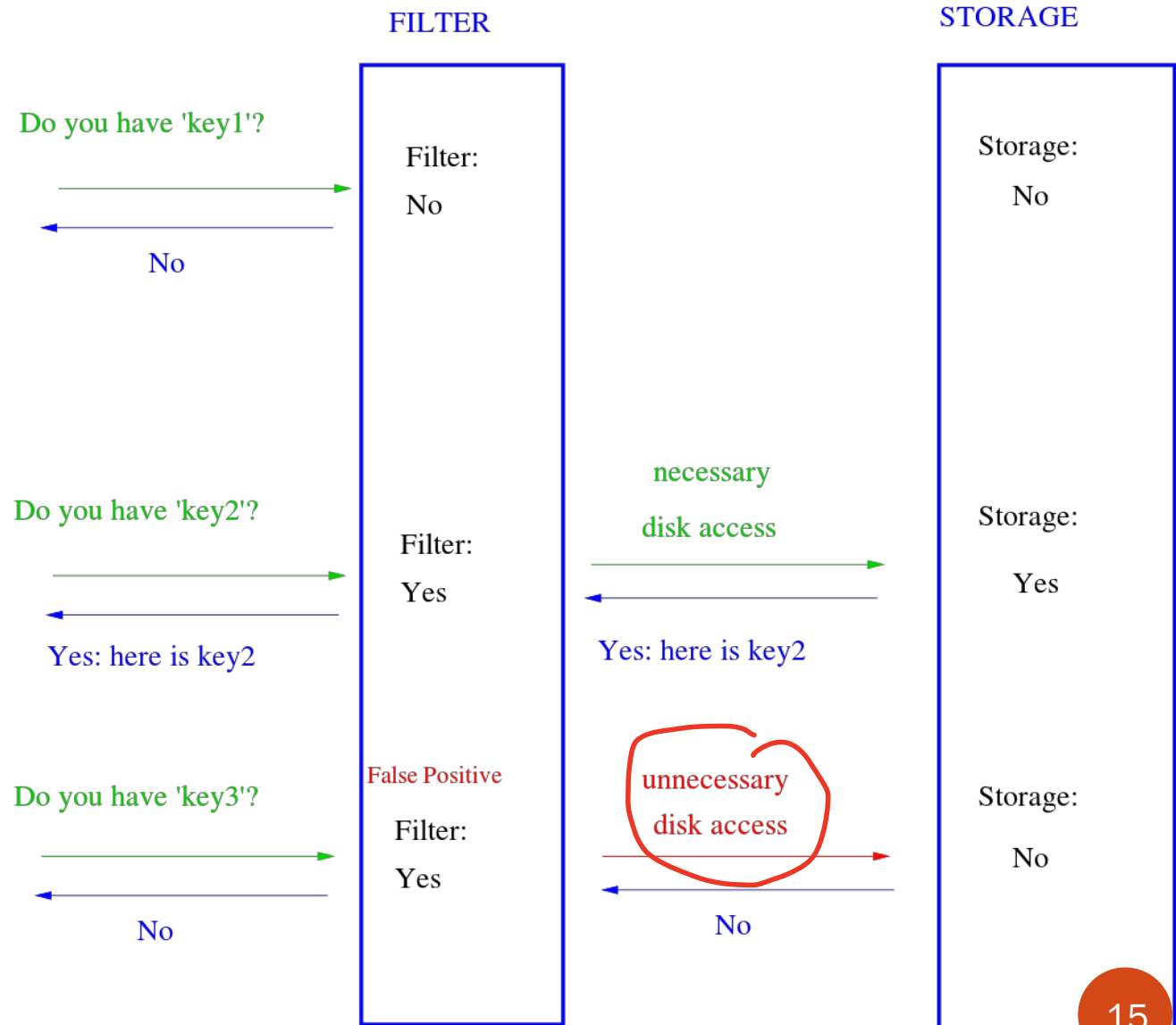
Parameter settings

- $|S| = 1$ billion, $n = 8$ billion
 - $k = 1: (1 - e^{-1/8}) = 0.1175$
 - $k = 2: (1 - e^{-1/4})^2 = 0.0493$
- Can we increase k to ∞ ?
 - No ☺
- Optimal value of k :
 - $k = n \ln(2) / m$
 - Our case, optimal $k = 8 \ln(2) \sim 6$
 - Error at $k = 6: (1 - e^{-3/4})^6 = 0.0216$



Storage system

- Bloom filter speeds up answers in a key-value storage system
- Values are stored on a disk which has slow access times while bloom filter decisions are much faster



Bloom filter: Summary

- Bloom filters guarantee no false negatives and use limited memory, which is suitable for processing data stream
 - Great for pre-processing before more expensive check
- Very suitable for hardware implementation
 - Hash function computation can be parallelized

low memory.

Homework

- Implement the Bloom filter algorithm on the dataset from <https://archive.ics.uci.edu/ml/datasets/bag+of+words>, using word tokens from `vocal.kos.text`:
 - **Description:** Consider that word tokens in the file as “good” words and words not mentioned in the file as “bad” words
 - **Goal:** Implement an add-on for Gmail to help users filter out “bad” words efficiently while writing an email